TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
HCM·UTE
HCMC University of Technology and Engineering

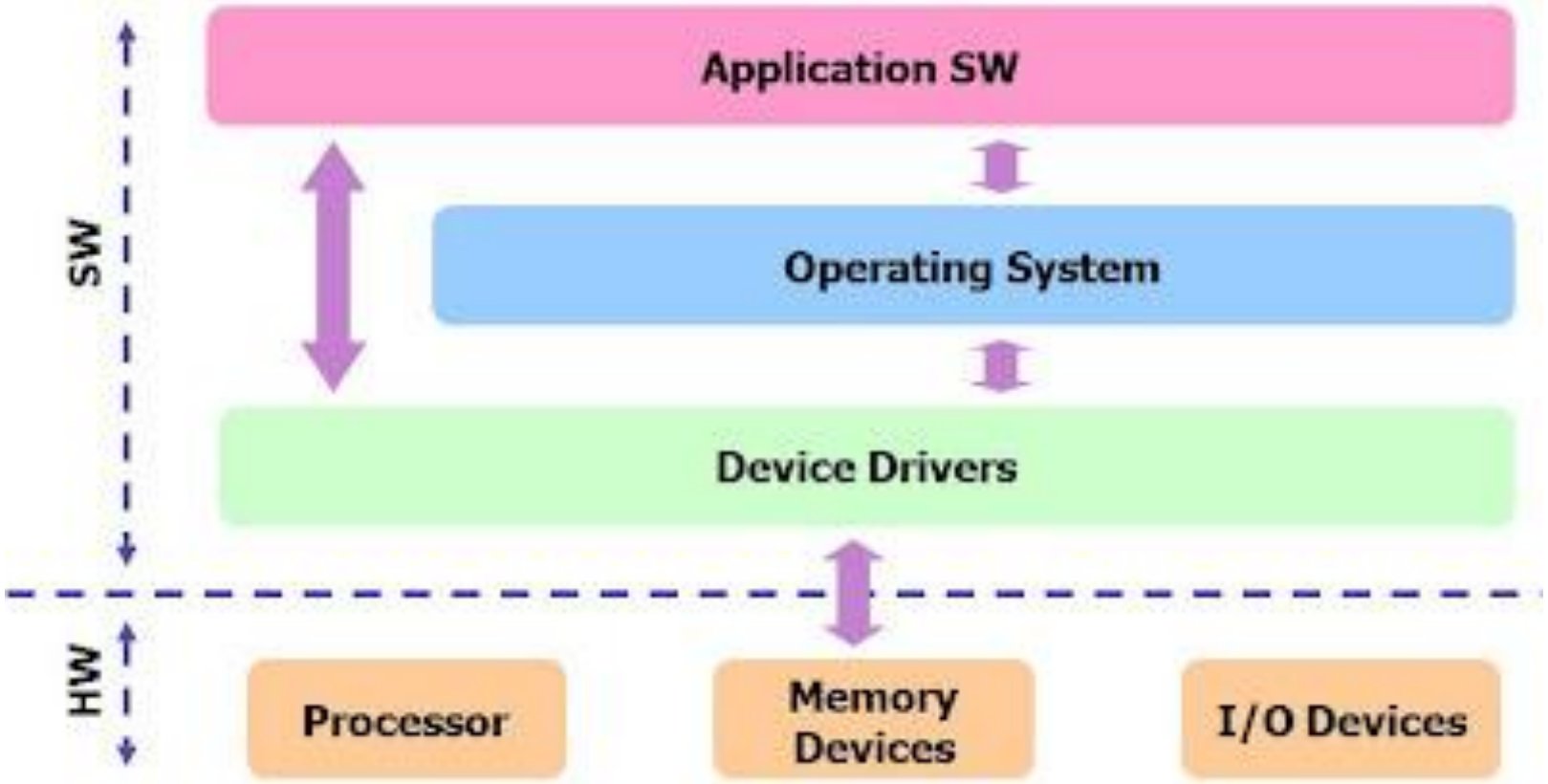
Chapter 2
Embedded System's Components



CONTENTS

- Components of Embedded System
- Simple Circuit of Embedded System
- Hardware of Embedded System
- Software of Embedded System
  - Device Driver
  - Embedded OS
  - Middleware and applications software

Different Layers of an Embedded System

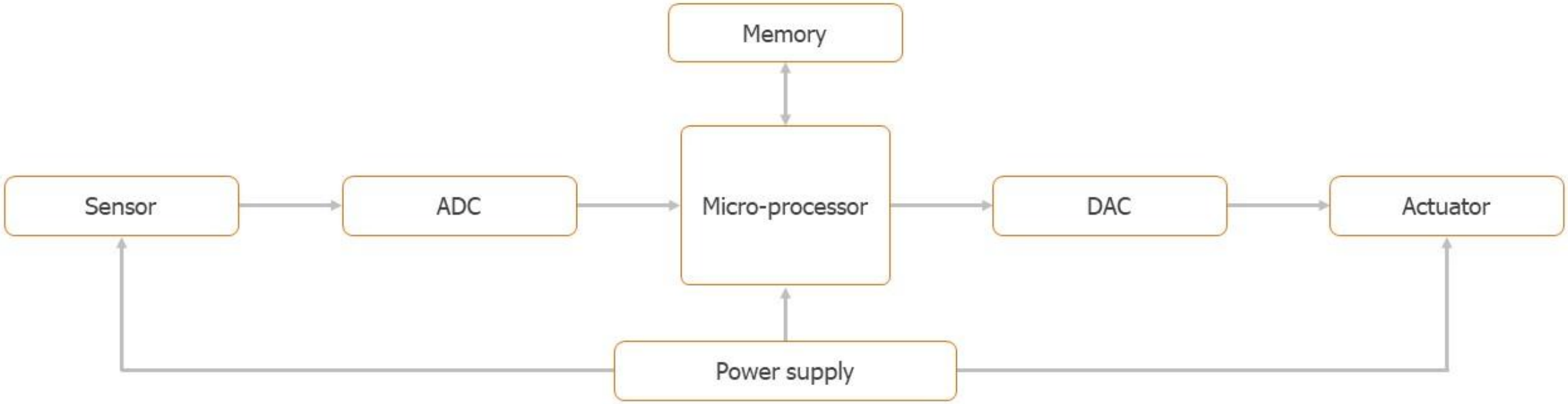


- Embedded hardware consists of:
  - The processor, which is the main system controller.
  - The memory devices, where instructions and data are stored.
  - I/O devices through which communication between the processor and the external world is enabled.



- Embedded software consists of:
  - Device drivers, which are software written to directly control the embedded hardware and provide an API interface to the upper software layers. Device drivers abstract hardware details from application programmers.
  - Operating system is software that manages the systems resources.
  - Application software is the software that performs the main function that the system was built for.

Block Structure Diagram of Embedded System



HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Hardware of Embedded System

Sơ đồ chuỗi tín hiệu phần cứng

Bước	Thành phần	Chức năng trong chuỗi tín hiệu
1	Cảm biến (Sensor)	Thu nhận đại lượng vật lý → tín hiệu điện tương tự (analog)
2	Bộ biến đổi A/D (ADC)	Chuyển tín hiệu tương tự → tín hiệu số
3	Phần xử lý thông tin (Processing Unit)	CPU/MCU xử lý dữ liệu số theo chương trình
4	Bộ nhớ (Memory)	Lưu chương trình và dữ liệu phục vụ xử lý
5	Hệ thống Bus (Bus System)	Kết nối, truyền dữ liệu/địa chỉ/tín hiệu điều khiển giữa các khối
6	Các module vào/ra (I/O Modules)	Giao tiếp với thiết bị ngoại vi, mạng, người dùng
7	Bộ chuyển đổi D/A (DAC)	Chuyển tín hiệu số → tín hiệu tương tự để điều khiển cơ cấu chấp hành
8	Bộ hiển thị (Display)	Hiển thị trạng thái/kết quả xử lý cho người dùng

Sensor



















Accelerometer





Touch Sensor (Capacitance based)



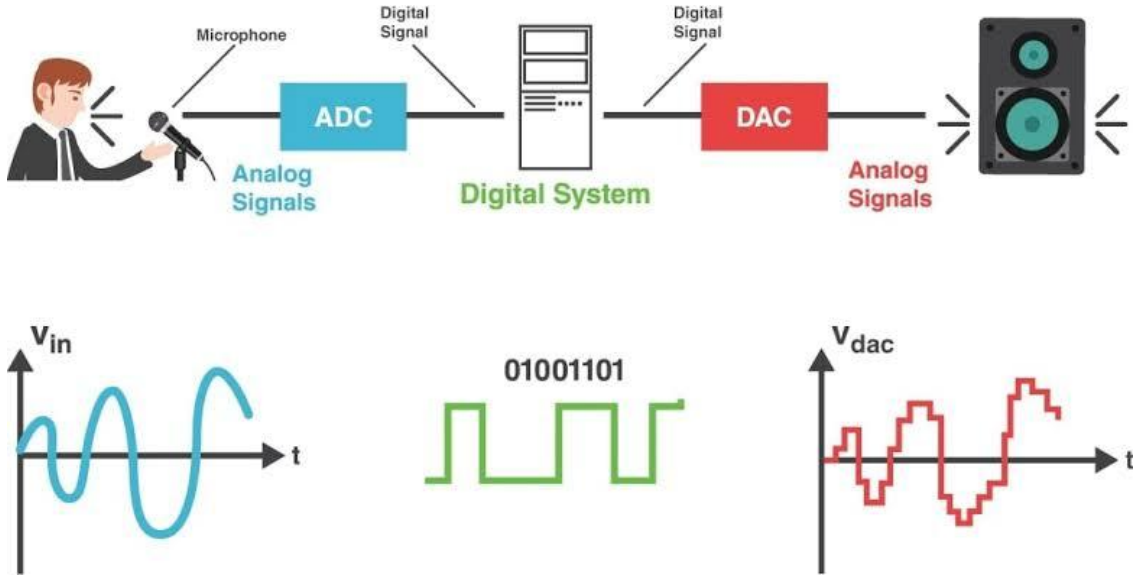
Sound sensor
Microphone

ADC

- Analog to Digital Conversion (i.e. ADC), as the name suggests, is all about converting a given analog signal into its digital form or say a digital value
- This conversion or measurement happens in presence of a fixed and accurate reference voltage
- The analog signal is compared to this reference voltage and then estimations are made to get the final measured value

Digital Signal Processor (DSP)

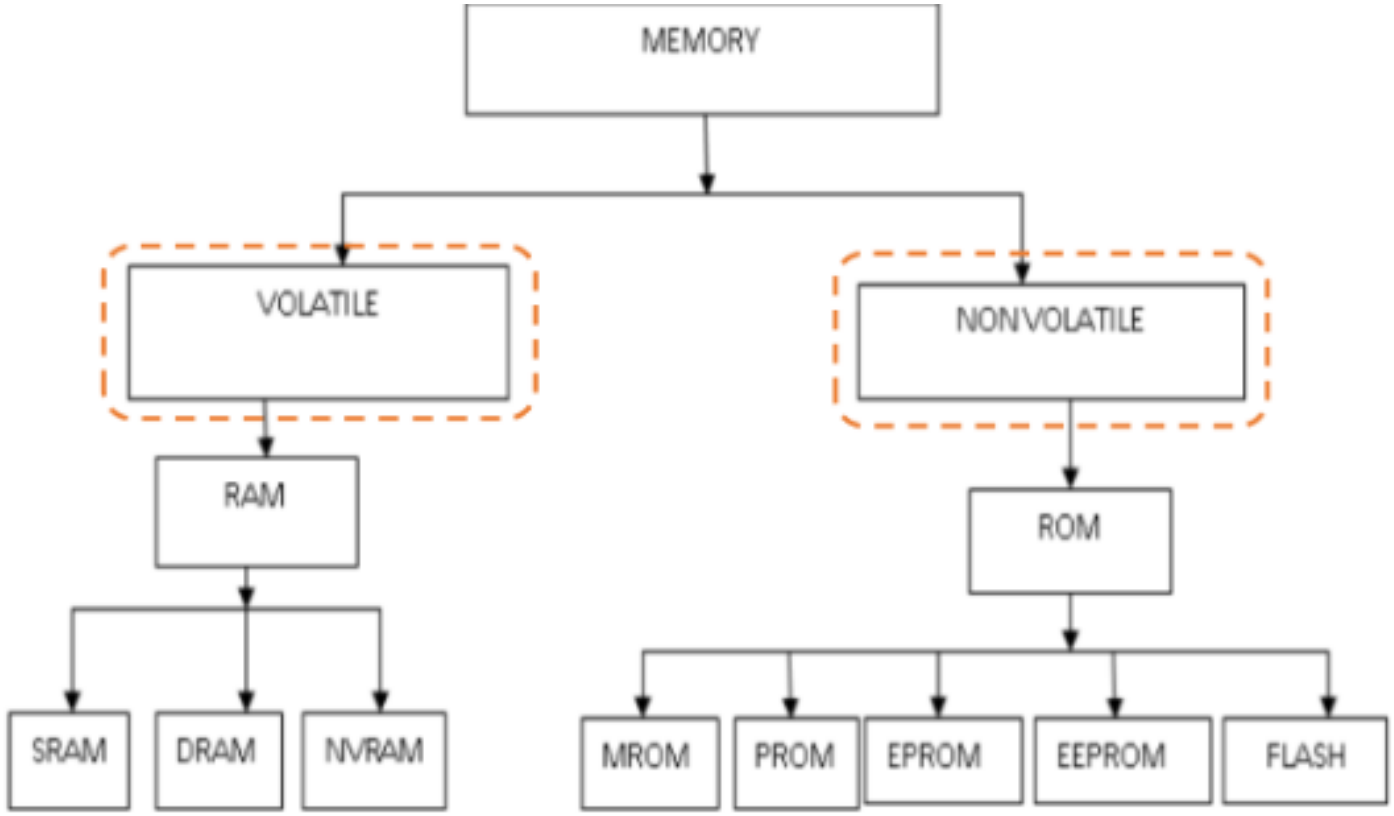
- The DSP is typically used in conjunction with a **general purpose processor** to perform specialized tasks such as image, speech, audio, video processing.
- The DSP is architected as a **Harvard** rather than the **von Neumann** machine





Memory

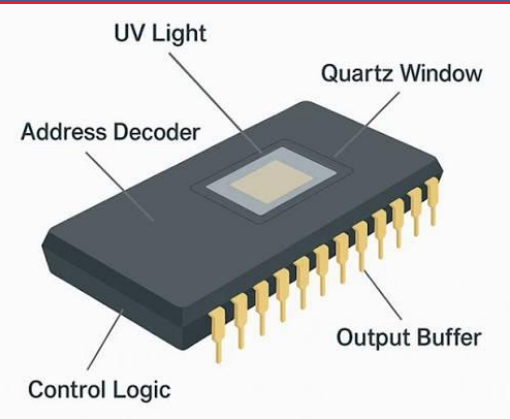
- On-Chip Memory



Memory

- On-Chip Memory
  - Generally, both primary and secondary memory can be divided into two groups, non-volatile or volatile. Non-volatile memory is memory that can store data after the main power source to the board has been shut off.
  - On embedded boards, there are:
    - Non-volatile: Real-Only Memory (ROM) and auxiliary memory (magnetic disks, magnetic tapes, and optical discs)
    - Volatile: Random-Access Memory (RAM)

Memory



- On-Chip Memory
  - Read-Only Memory (ROM)
    - **MROM (mask ROM):** that is permanently etched into the microchip during the manufacturing of the processor, and cannot be modified later
    - **PROM (programmable ROM),** (one-time programmables – OTPs): that can be integrated on-chip, that is one-time programmable by programmer
    - **EPROM (erasable programmable ROM):** can be erased and reprogrammed more than once (depends on the processor), is erased by intense ultraviolet light
    - **EEPROM (electrically erasable programmable ROM):** is cheaper and faster variation of the EEPROM is Flash memory

Memory

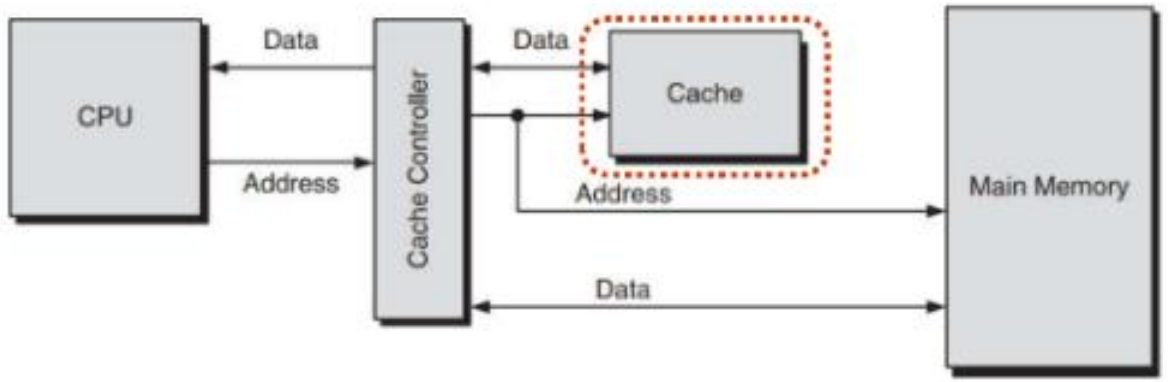
- On-Chip Memory
  - Random-Access Memory (RAM)
    - DRAM (dynamic RAM): using a stored charge mechanism, which can leak away, hence DRAM needs to be refreshed periodically. It is used for larger memory systems.

      → This makes DRAM more complex and power-consuming. However, as this form of memory is less expensive to produce compared to SRAM
    - SRAM (static RAM): is designed with bit storage implemented using a latch-type mechanism (stored data does not have to be refreshed)

      → It is used for higher speed memory systems because they are faster than DRAM designs. It is capable of running at 133MHz, a typical clock frequency

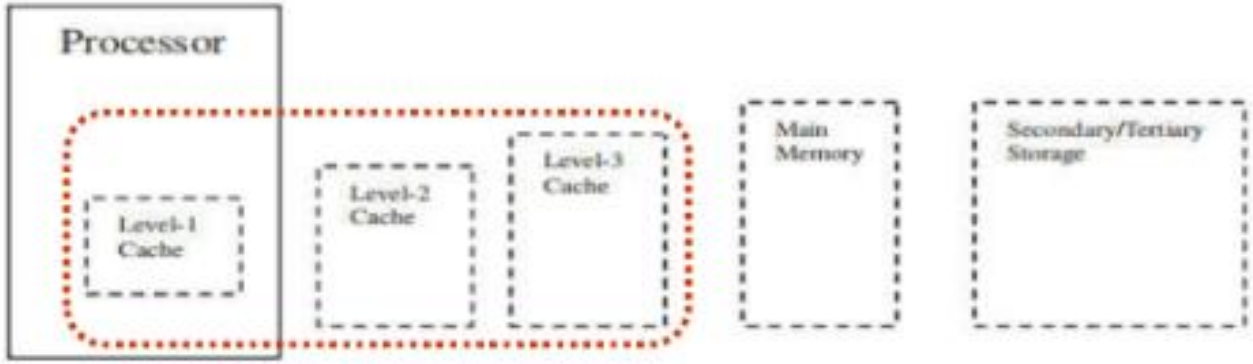
Memory

- On-Chip Memory
  - Cache Memory
    - When program execution needs an instruction or data that is not in the cache, it must be brought in from main memory (typically using **DRAM**)
    - The goal is to operate out of cache memory – to the greatest extent possible. Caching requires a certain amount of higher speed memory (typically **SRAM**)



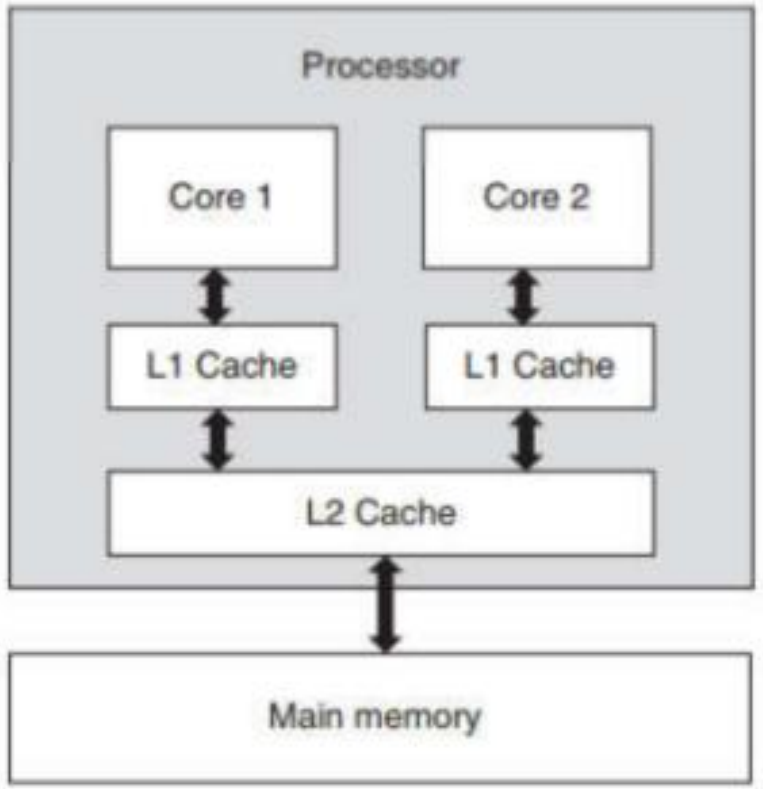
Memory

- On-Chip Memory
  - Cache Memory
    - CPUs have multilevel cache memory. They are referred to as L1 for level one, L2 for level two, L3 for level three,… A CPU directly works with L1 cache.
    - The goal of the L1 cache is to line up with the CPU so that it has data to work on every processor cycle at the best. It is a small, high-speed cache incorporated right onto the processor's chip



Memory

- On-Chip Memory
  - Cache Memory
    - The L1 cache typically ranges in size from 8 to 64 KB and uses the high-speed SRAM
    - The L2 cache is expected to feed data to the L1 cache every few processor cycles, while L3 cache can feed data to the L2 cache at a further slower rate
    - The three levels exchange data based on CPU requirements.



Board Buses

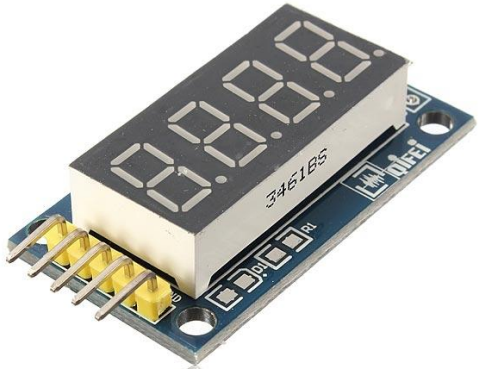
- A bus's performance is typically measured by **its bandwidth**, the amount of data a bus can transfer for a given length of time.
- The actual physical design of the but (its length, the number of lines, the number of supported devices, and so on) **limits or enhances** its performance
- The shorter the bus, the fewer connected devices, and the **more data lines**, typically **the faster** the bus and the higher **its bandwidth**

I/O Module

- Processor I/O components
  - **Networking and communications I/O**
  - **Input** (keyboard, mouse, remote control, voice, etc.)
  - **Graphics and output I/O** (touch screen, CRT, printers, LEDs, etc)
  - **Storage I/O** (optical disk controllers, magnetic disk controllers, magnetic tape controllers, etc.)
  - **Debugging I/O** (BDM, JTAG, serial port, parallel port, etc,…)
  - **Real-time and miscellaneous I/O** (Timers/counters, ADC and DAC, key, switches, and so on)

Display Unit









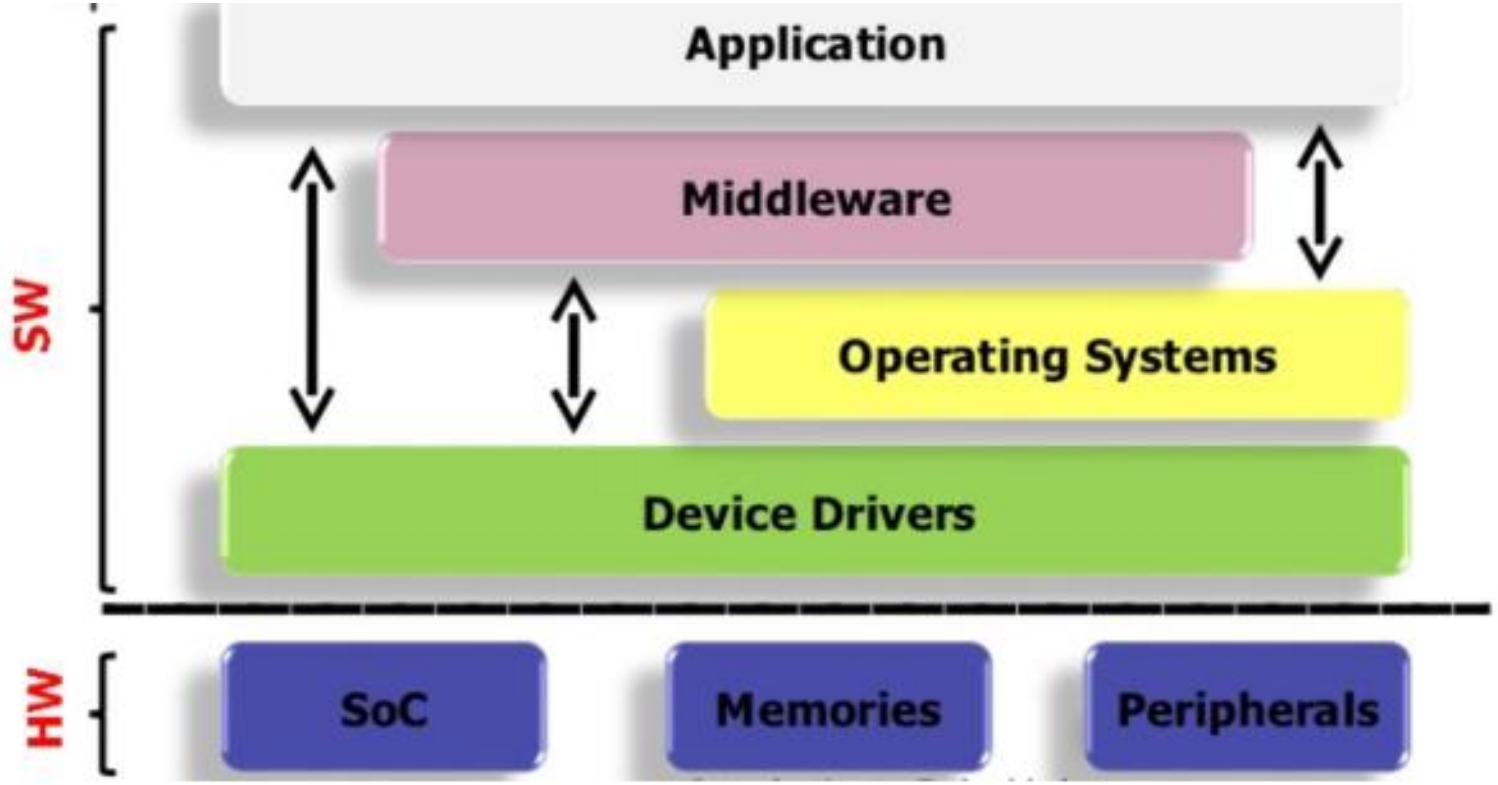


01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

4. Software of Embedded System

4.1. Overview

- Device drivers
- Real-time operating system
- Middleware and application software



4.1. Overview

- Basically, Embedded Software can be divided into 2 general classes:
  - **System software**
    - Device Drivers;
    - Operating Systems;
    - Middleware;
  - **Application software** is the **upper-level software** that defines the function and purpose of the embedded device and which handles most of the interaction with **users** and **administrators**.

→ All embedded systems that require software have, at the very least, **device driver software** in their **System software** layer

4.2. Device Drivers

- The software that directly interfaces with and controls this hardware is call **a device driver**
- **Device drivers** are the software libraries that **initialize** the hardware, and **manage** access to the hardware by higher layers of software.
- **Device drivers** are the linkage between the hardware and the **Operating system, Middleware, and Application Layers**

4.2. Device Drivers

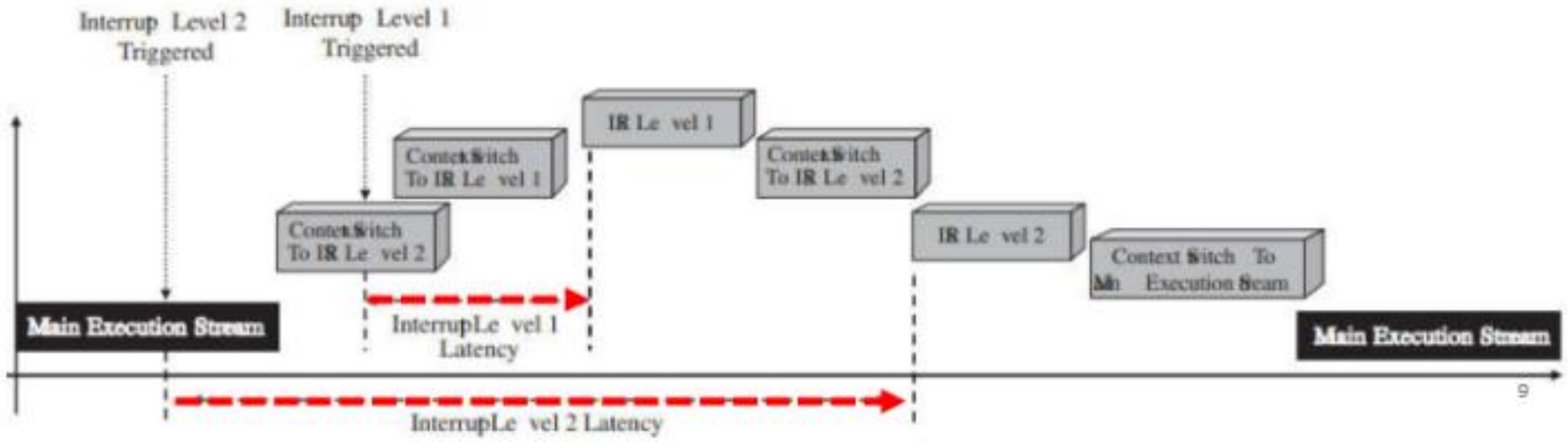
- **Device drivers** are typically considered: **Architecture-specific** or **Generic**
  - **Architecture-specific drivers** initialize and enable components **within a master processor** include on-chip memory, integrated memory managers
  - **Generic driver** manages hardware that is located on the board and not integrated onto the master processor: **off-chip memory** (controllers, level-2 cache, Flash, etc…), **off-chip I/O** (Ethernet, RS-232, display, mouse, etc,…), **board buses** (I2C, PCI, CAN, etc,…)
- **Each of driver function** typically has code that interfaces directly **to the hardware** and code that interfaces to **higher layers of software**

4.2. Device Drivers

- **Device drivers for Interrupt-Handling**
  - Interrupts are signals (*triggered by events*): cause the master processor to stop executing the current instruction stream and start the interrupt (**IR**) process.
    - **Software interrupts** are explicitly triggered internally by some instruction within the current instruction stream being executed by the master processor
    - **Internal hardware interrupts** are initiated by an event that is a result of a problem with the current instruction stream that is being executed by the master processor**: illegal math operations** (overflow, divide-by-zero), **debugging** (single-stepping, breakpoints), **invalid instruction**,…
    - **External hardware interrupts** are initiated by **hardware other.**

4.2. Device Drivers

- **Device drivers for Interrupt-Handling**
  - The **performance** of an embedded design is affected by the **latencies** (delays) involved with the interrupt-handling scheme. **The figure** summarizes the variables that impact **interrupt latency.**



4.2. Device Drivers

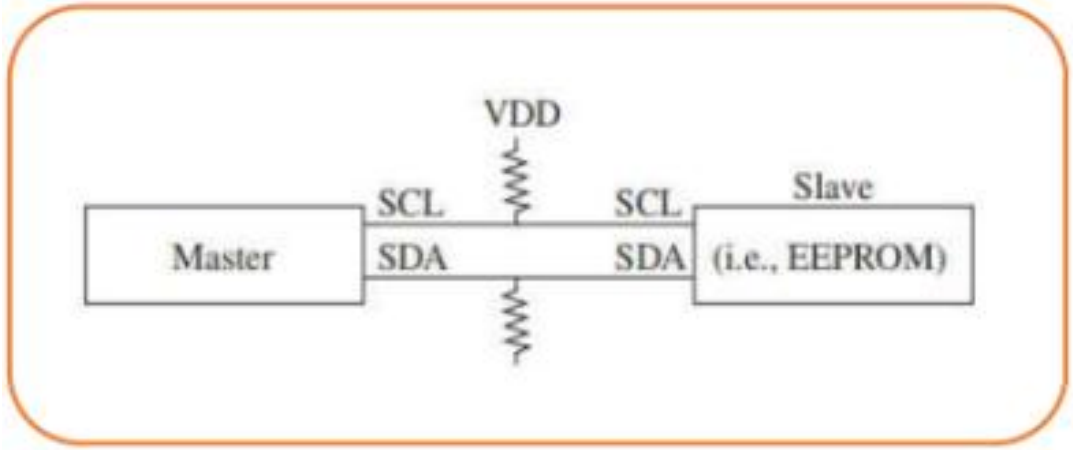
- **Memory Device Drivers**
  - The software must **provide the processors** in the system with the ability to access various portions of the memory map.
  - The software involved in **managing the memory** **on the master processor** and **on the board**, as well as managing **memory hardware** mechanisms, consists of the **device drivers** for the management of the overall **memory subsystem**
  - The **memory subsystem** includes all types of memory management components, such as **Memory controllers** and Memory Management Unit (**MMU**), as well as the types of memory in the memory map, such as **registers, cache, ROM, RAM,…**

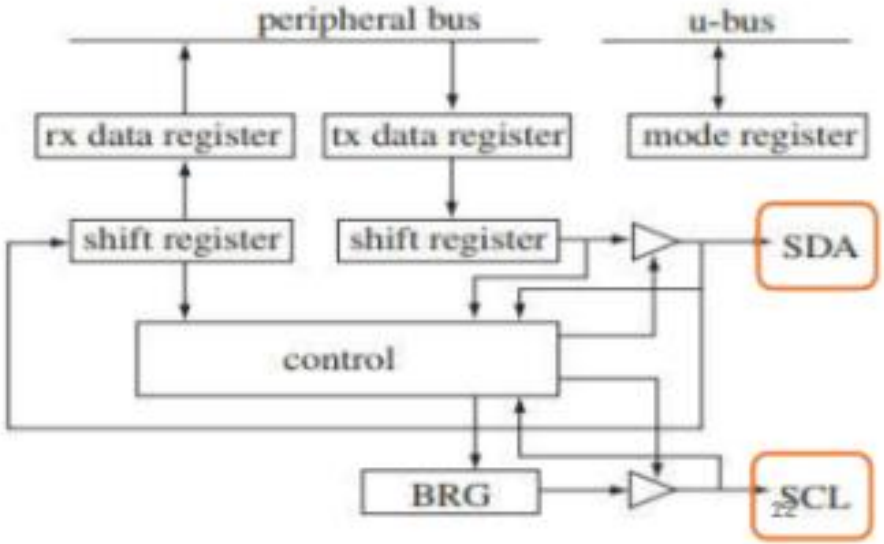
4.2. Device Drivers

- **On-board Bus Device Drivers**
  - Some type of **protocol** that defines how devices gain **access to the bus**
  - The **rules** attached devices must follow **to communicate over the bus**
  - The **signals** associated with the **various bus lines**

4.2. Device Drivers

- **On-board Bus Device Drivers**
  - **An example:** I2C Protocol (Inter-Integrated Circuit) is a serial bus with one serial data line (SDA-Serial Data) and one serial clock line (SCL-Serial Clock)



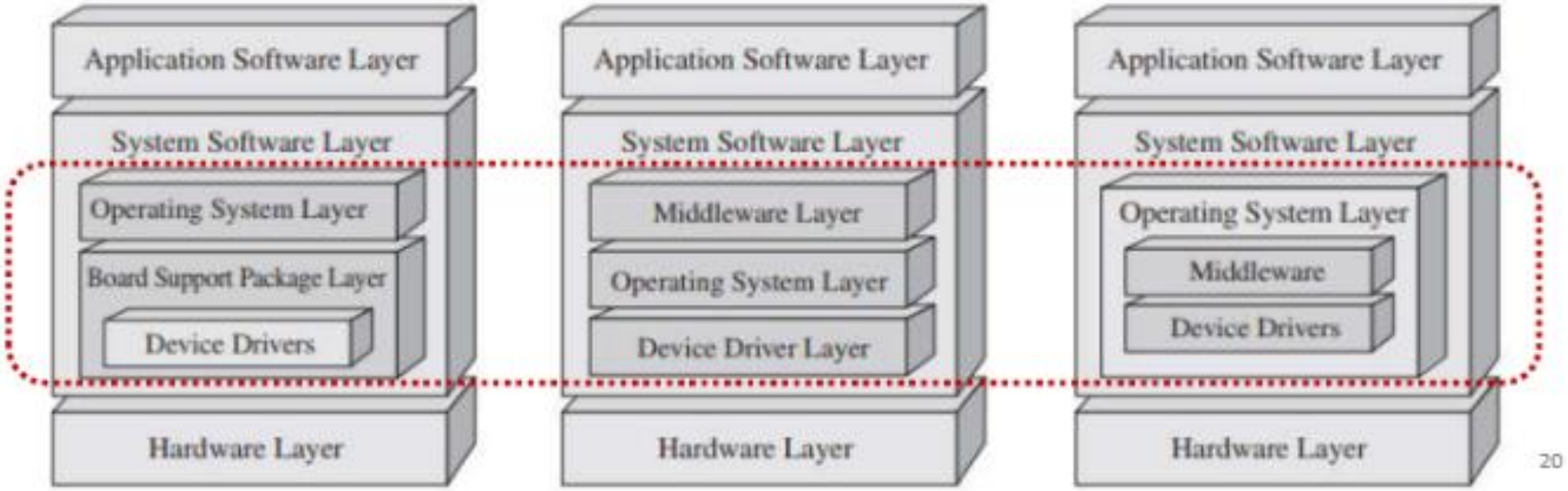


4.2. Device Drivers

- **Board I/O Drivers**
  - The board I/O subsystem components that require some form of **software management** include the component integrated on the **master processor**, as well as an **I/O slave controller**, if one exists.
  - The **I/O controllers** have a set of status and control registers used to control the processor and check on its status.

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - An **operating system (OS)** is **an optional part** of an embedded device's system **software** stack.

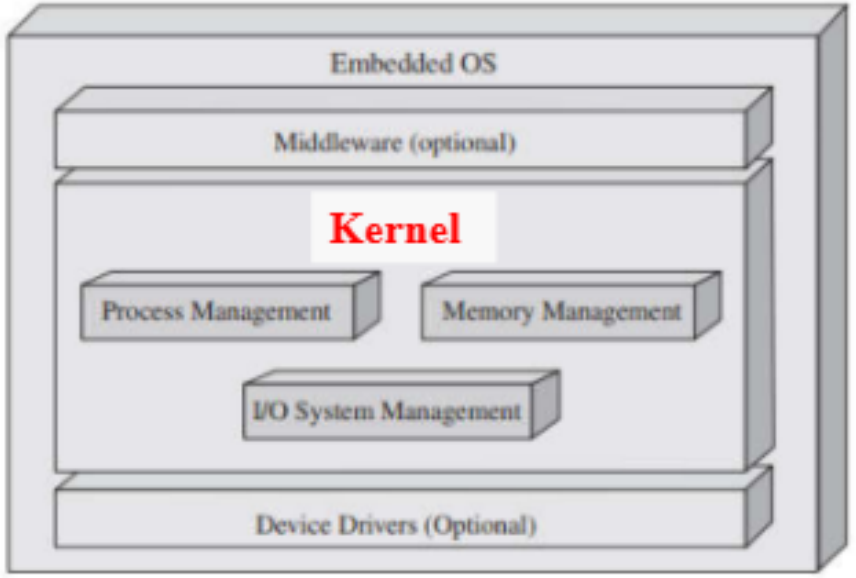


4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - The OS is a set of software libraries that serves **two main purposes:**
    - Providing an abstraction layer for software on top of the OS (**be less dependent on hardware**) → Making the development of **Middleware** and **Applications** that sit on top of the OS easier
    - Managing the various system hardware and software resources → **Ensure** the entire system **operates efficiently** and **reliably**

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - An OS is composed of multiple software components, and the core components in the OS form its kernel.
  - The kernel always runs in system mode, while other parts and all applications run in user mode
  - All Oses have a kernel at the very least. The kernel is a component that contains the **main functionality** of the OS (**Figure**)



- Process Management
- Memory Management
- I/O System Management

22

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

1. Process Management

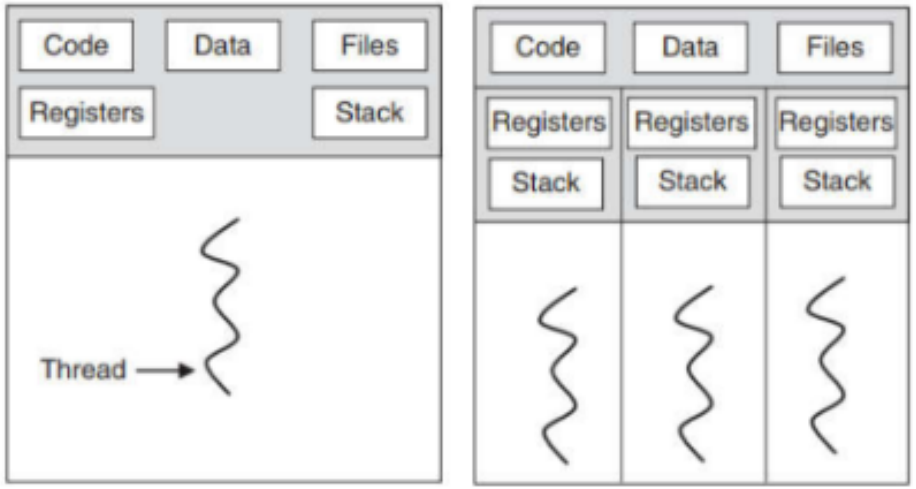
- A process is an instance of a program in execution. It is a unit of work within the system. Starting a new process is a heavy job for the OS, which includes allocating memory, creating data structures, and coping code.
- Sometimes, processes need to communicate with each other while they are running. This is called Inter Process Communication (IPC). The OS provides mechanisms to support IPC

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:
    1. **Process Management**
       - A thread is a path of execution within a process and the basic unit to which OS allocates processor time. A process can be single-threaded or multi-threaded
       - Theads are used for small tasks, whereas processes are used for more heavyweight tasks. Therefore, a thread is often called a lightweight process.
       - A thread is the smallest unit of work managed independently by the scheduler of the OS
       - In RTOSs, the term tasks is often used for threads or single-threaded processes

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:
    1. **Process Management**
       - Theads use far less resources compared to processes. However, each thread of the same process has its own thread status, program counter, registers, and stack



4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

2. Memory Management

- Physical address
  - Main memory is the most critical resource in a computer system in terms of speed at which programs run
  - Each memory location has a physical address. In most computer architectures, memory is byte-addressable, meaning that data can be accessed 8 bits at a time.
  - In general, only system software such as Basic Input/Output System (BIOS) and OS can address physical memory

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

2. Memory Management

- Logic addresses
  - Most application programs do not have knowledge of physical addresses. Instead, they use logic address
  - A logical address is the address at which a memory location appears to reside from the perspective of an executing application program
  - A logical address may be different from the physical address due to the operation of an address translator or mapping function.

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

2. Memory Management

- Virtual memory
  - In a computer supporting virtual memory, the term physical address is used mostly to differentiate from a virtual address
  - In particular, in computers utilizing a memory management unit (MMU) to translate memory addresses, the virtual and physical addresses refer to an address before and after translation performed by the MMU, respectively
  - **Why using the virtual memory?**

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

2. Memory Management

- Virtual memory
  - Protecting memory by Virtual Memory: If two or more processes are running at the same time and use direct addresses, a memory error in one process could destroy the memory being used by the other process, taking down multiple programs due to a single crash

→ The virtual memory technique, on the other hand, can ensure that each process is running in its own dedicated address space.

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

2. Memory Management

- Memory Allocation: When a process is started, the OS allocates memory to the process and then loads module from disk into the memory allocated to the process.
- The simplest memory management technique is Single contiguous allocation: all the memory is made available to a single application
- Another technique is Partitioned allocation: It divides memory into multiple memory partitions; each partition can have only one process → The memory manager selects a free partition and assigns it to a process when the process starts and deallocates it when the process is finished

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

2. Memory Management

- Paged Allocation: divides memory into fixed-size units called page frames and the program's virtual address space into pages of the same size. The memory management unit (MMU) maps pages to frames
- The physical memory can be allocated on a page basis while the address space appears to be contiguous. Paging does not distinguish and protect programs/data separately

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

2. Memory Management

- Segmented memory: allows programs/data to be broken up into logically independent address spaces and to aid sharing and protection
- Segments are areas of memory that usually correspond to a logical grouping of information such as a code procedure or a data array.

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

3. Interrupts Management

- An interrupt is a signal from a device attached to a computer or from a running process within the computer → indicating an event that needs immediate attention
- The processor responds by suspending its current activity, saving its state and executing a function called an Interrupt Handler (Interrupt Service Routine, ISR) to deal with the event

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

4. Multitasking

- Real-world events may occur simultaneously. Multitasking refers to the capability of an OS that supports multiple independent programs running on the same computer
- It is mainly achieved through time-sharing, which means that each program uses a share of the computer's time to execute, which is addressed by schedulers, follow scheduling algorithms
- It involves managing both threads and processes, which can run simultaneously, and independently

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

4. Multitasking

- Process-based Multitasking (Multi-processing)
  - A form where multiple processes running concurrently
  - Each process has its own memory space/resources, and runs independently from others
  - Processes are heavyweight due to the overhead of managing separate memory and context switching

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

4. Multitasking

- Thread-based Multitasking (Multi-threading)
  - Each thread is lightweight
  - Threads share the same address space
  - Inter-thread communication is fast and low-cost

Benefits: → Using better CPU Utilization
→ Reduced Response Time
→ Handling Multple Clients simultaneously

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

4. Multitasking

- An example of Multi-thread



4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

5. File system Managament

- Files are the fundamental abstraction of secondary storage devices. Each file is a named collection of data stored in a device
- An important component of an OS is the file system, which provides capabilities of file management, auxiliary storage management, file access control, and integrity assurance
- File management is concerned with providing the mechanisms for files to be stored, referenced, shared and secured

4.3. Embedded Operating Systems

- General-Purpose Operating Systems
  - There are some of the main functions of a typical general-purpose OS:

6. I/O Management

- Modern computers interact with a wide range of I/O devices: keyboards, mice, printers, disk drives, USB drives, monitors, networking adapters, and audio systems,…
- One purpose of an OS is to hide features of hardware I/O devices from the user. The OS communicates with the I/O device through the device controller (drivers)
- I/O operation often have high latencies. Most of this latency is due to the slow speed of peripheral devices.

4.3. Embedded Operating Systems

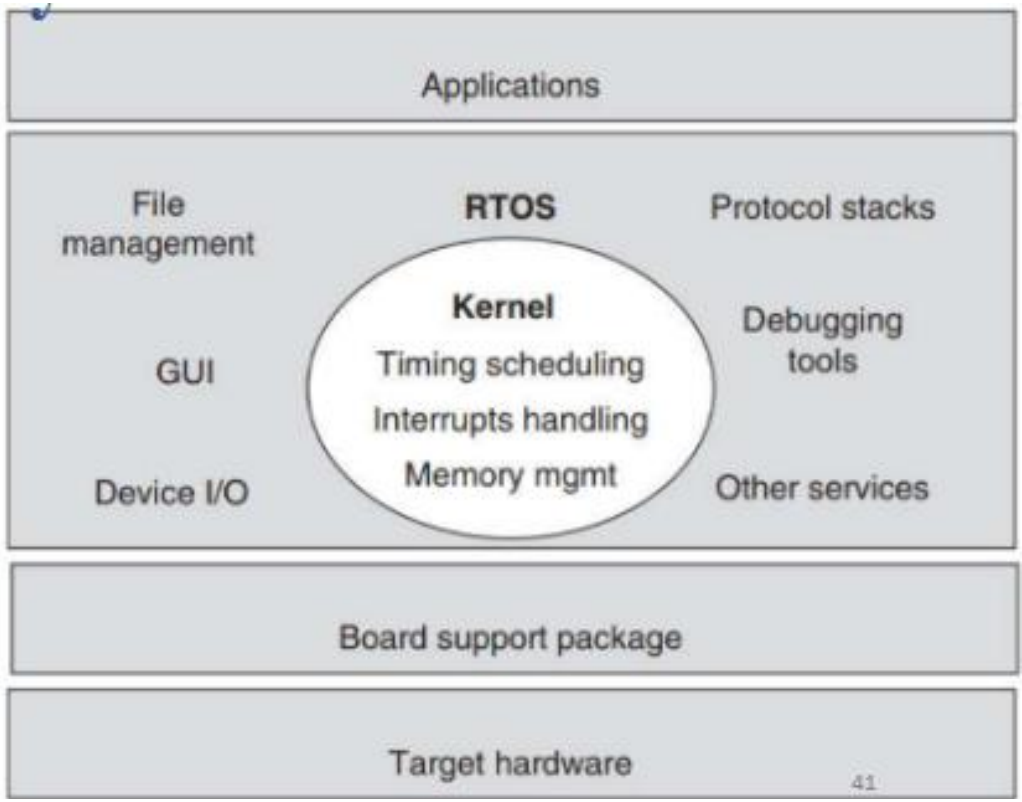
- Real-Time Operating System
  - Althrough a general-purpose OS provides a rich set of services it takes too much space and contains too many functions that may not be necessary for a specific real-time application
  - Moreover, it is not configurable, and its timing uncertainty offers no guarantee to system response time.

→ Therefore, a general-purpose OS is not suitable for Real-time Embedded Systems

4.3. Embedded Operating Systems

- Real-Time Operating System
  - An RTOS generally constains a **Real-time kernel** (Figure) and other higher-level services: file management, protocol stacks, a Graphical User Interface (GUI) and other components

  → Most additional services **involve around** I/O devices.



4.3. Embedded Operating Systems

- Real-Time Operating System
  - The heart of many computerized embedded systems is Real-time Operating System (RTOS)
  - An RTOS is an OS that supports the construction of applications that must meet real-time constraints and obtain correct computation results
  - A real-time kernel is software that manages the time and resources of a microprocessor or microcontroller and provides important services such as task scheduling and interrupt handling to applications

4.3. Embedded Operating Systems

- Real-Time Operating System
  - There are 3 key requirements of RTOS design:
    - The timing behavior of the OF must be predictable. For all services provided by the OS, the upper bound on the execution time must be known
    - The OS must manage the timing and scheduling, and the scheduler must be aware of task deadlines
    - The OS must be fast. A fast RTOS helps take care of soft real-time constraints of a system as well as guarantees hard deadlines

4.3. Embedded Operating Systems

- Real-Time Operating System
  - There are some most important real-time services for RTOS kernel:
    1. Clocks and Timers:
       - Most embedded systems must keep track of the passage of time. The length of time is represented by the number of system ticks in most RTOS kernels.
       - The RTOS works by setting up a hardware timer to interrupt periodically, every millisecond, and bases all timings on the interrupts
       - Based on the system tick, an RTOS kernel allows you to call functions of your choice after a given number of system ticks
       - Timers improve the determinism of real-time applications and allow applications to set up events at predefined intervals or time.

4.3. Embedded Operating Systems

- Real-Time Operating System
  - There are some most important real-time services for RTOS kernel:

2. Priority Scheduling

- Because real-time tasks have soft or hard deadlines, they differ in terms of urgency. Therefore, tasks in an RTOS are typically assigned different priority levels.
- To ensure that a higher-priority task is executed before its deadline: if a higher-priority task becomes ready while the processor is executing a lower-priority task, the RTOS will temporarily suspend the lower-priority task and immediately schedule the higher-priority one on the processor.

This mechanism is called **preemption**.

4.3. Embedded Operating Systems

- Real-Time Operating System
  - **2. Priority Scheduling**
    - Task scheduling in real-time applications is generally **priority-based** and **preemptive**.
    - In priority-driven preemptive scheduling, the scheduler uses a **clock interrupt task** that allows it to switch tasks after a certain amount of execution time has passed.
    - The performance of priority-driven scheduling depends heavily on **how priorities are assigned**. However, this approach can cause **low-priority tasks to starve** and miss their deadlines.

4.3. Embedded Operating Systems

- Real-Time Operating System

3. Intertask Communication and Resource Sharing:

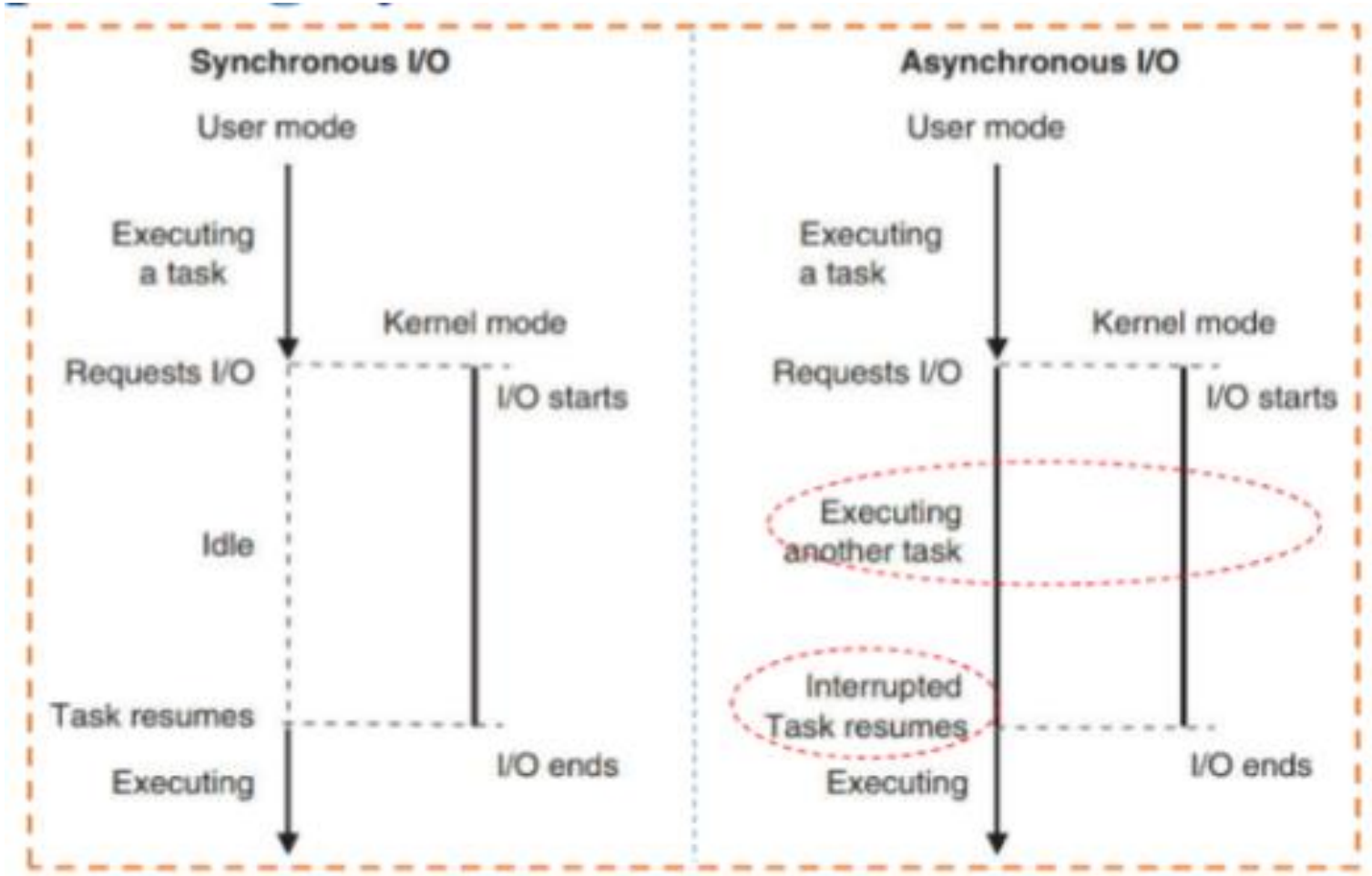
- In an RTOS, one task cannot directly call another. Instead, tasks exchange information through **message passing** or **shared memory**, and they coordinate their execution and access to shared data using **real-time signals**, **mutexes**, or **semaphore objects**.
- **Real-time signals** (similar to software interrupts):
  - A signal is automatically sent to the parent process when a child process terminates.
  - Signals are also used for various **synchronous** and **asynchronous** notifications.

4.3. Embedded Operating Systems

- Real-Time Operating System
  - **4. Asynchronous I/O**
    - There are two types of I/O synchronization:
    - In **synchronous I/O**, when a user task requests an I/O operation and the request is granted, the system **waits until** the operation is fully completed **before** it can continue processing other tasks. → This approach is suitable when the I/O operation is **fast and simple to perform**.
    - In **asynchronous I/O**, after a task requests an I/O operation, **while the task is waiting for the I/O to finish**, other tasks that **do not depend on the I/O result** can be scheduled and executed. → This helps improve **throughput**, **latency**, and **responsiveness**.

4.3. Embedded Operating Systems

- Real-Time Operating System
  - **4. Asynchronous I/O**



4.3. Embedded Operating Systems

- Real-Time Operating System

4. Memory Locking

- Memory locking is a real-time capability designed to help a process avoid the latency caused by fetching memory pages. It works by locking specific memory pages so they remain resident in physical memory.
- → This ensures that critical parts of an application stay in main memory, providing fine-grained control and reducing overhead.
- For example, memory locking can be used to keep a thread in memory that monitors a critical process requiring immediate attention.

4.4. Middleware and Application Software

Middleware is software that has been separated from the application layer for several reasons:

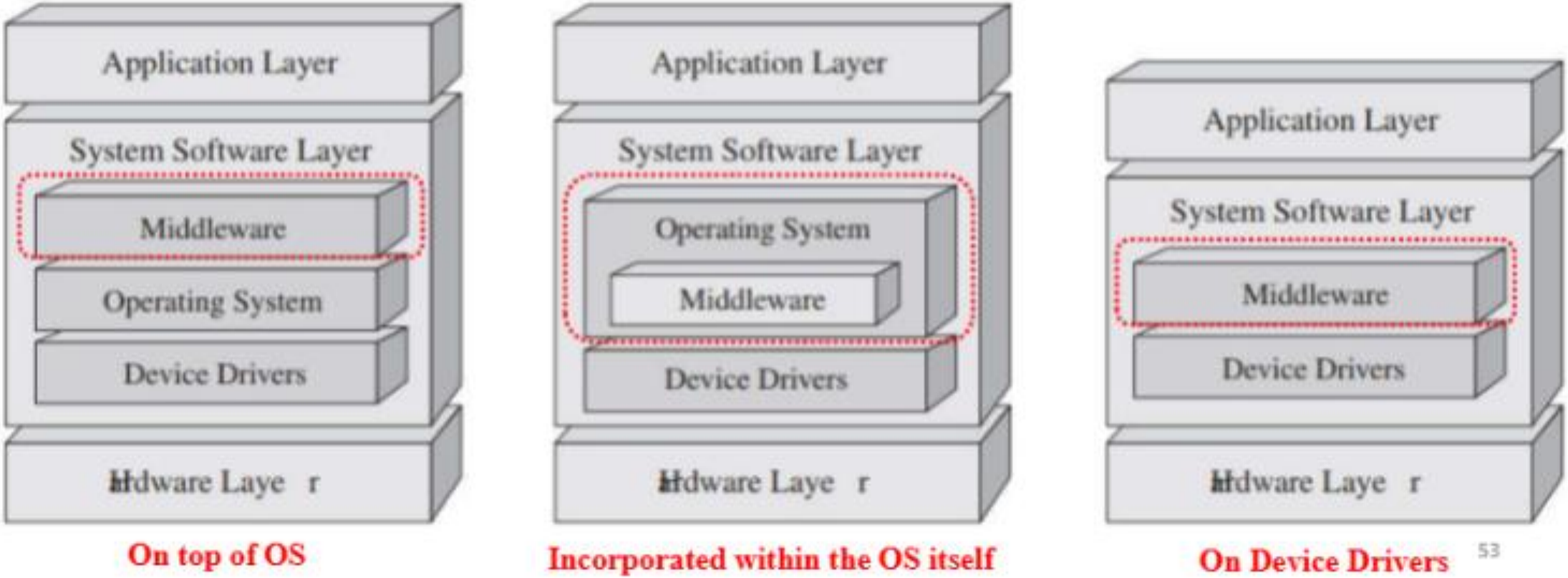
- One reason is that it may already be included as part of an off-the-shelf operating system package.
- Other reasons for removing it from the application layer include:
  - Enabling **reusability** across multiple applications,
  - **Reducing development cost or time** by purchasing it from a third-party vendor,
  - **Simplifying** the application's code.

4.4. Middleware and Application Software

- Middleware
  - Middleware is any system software that is not part of the OS kernel, device drivers, or application software.
  - In an embedded system, middleware is system software that typically sits on top of the device drivers or the operating system, and in some cases, it may even be integrated within the OS itself.

4.4. Middleware and Application Software

- Middleware



4.4. Middleware and Application Software

- Middleware
  - Middleware is software that acts as an intermediary between application software and the kernel or device drivers.
  - It can also serve as a layer that enables communication and coordination among different application programs. → In embedded systems with two or more applications, middleware provides an abstraction layer that supports **flexibility**, **security**, **portability**, **connectivity**, **intercommunication**, and **interoperability** between applications. → In short, middleware influences the embedded system across all layers.

4.4. Middleware and Application Software

- Middleware
  - There are some types of common middleware:
    1. General-purpose, meaning they are typically implemented in a variety of devices, such as networking protocols above the device driver layer and below the application layers of the OSI model, file systems, or some virtual machines.
    2. Market-specific, meaning they are unique to a particular family of embedded systems (such as a digital TC standard-based software that sits on an OS)
       - Proprietary, meaning it is closed software supported by a company that licenses it to others for use
       - Open, meaning it is standardized by some industry committee and can be implemented and/or licensed by any interested party

4.4. Middleware and Application Software

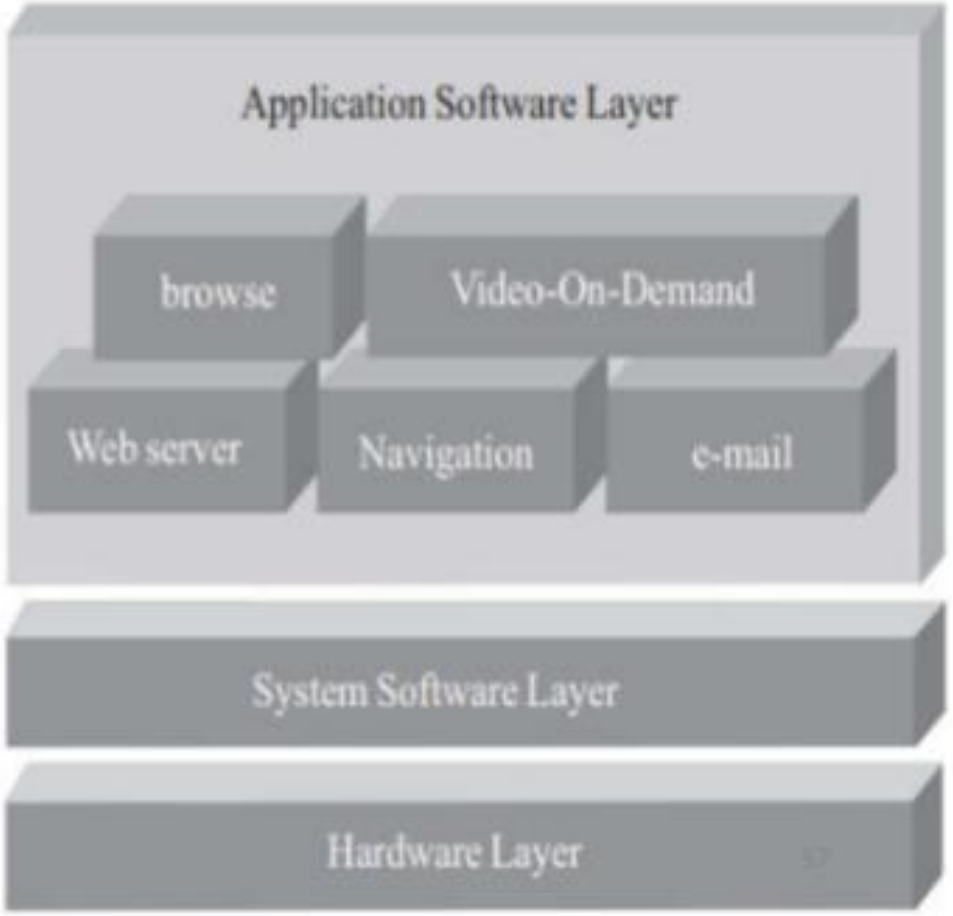
- Application Software
  - Application software sits on top of the system software layer, and is dependent on, managed, and run by the system software.
  - Application software is the software within the application layer that gives the device its character, differentiating it from other embedded devices.

→Like embedded standards, Embedded Application can be divided:

- Market specific (Implemented in only a specific type of device, such as video-on-demand applications in a interactive digital TV)
- General-purpose (Implemented across various types of devices)

4.4. Middleware and Application Software

- Examples
  - File Transfer Protocol Client Application (browsers or MP3 app,…)
  - Simple Mail Transfer Protocol and E-Mail (email)
  - Hypertext Transfer Protocol Client and Server (Web Server)



TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
HCM·UTE
HCMC University of Technology and Engineering

THANK YOU

